ELEC 475 Lab 2 Report

ELEC 475 Lab 2: From LeNet-5 to AlexNet

Field	Value
Course	ELEC 475: Computer Vision with Deep Learning, Fall 2026
Lab	Lab 2 Post-Lab: Full AlexNet, Normalization Study and Data Augmentation
Name	**Yahya Moahmed, Noel Steves**
Student number	**20355353, 20370449**
GitHub repository	**[ADD: GitHub repo URL]**
Hardware	**cuda (NVIDIA GeForce RTX 5060 Laptop GPU)**
Submitted	October 22, 2026

This report traces the evolution from LeNet-5 to a 5-stage AlexNet on 64×64 RGB input and measures the effect of each change on CIFAR-10 and Tiny ImageNet-200. The best model, AlexNet-64 with BatchNorm, dropout and data augmentation, reached 81.05 ± 0.52% final-epoch validation accuracy on CIFAR-10 (82.86 ± 1.07% at its best epoch), 26.6 points above Base LeNet-5, and 28.92% on Tiny ImageNet-200. All runs use Adam (lr = 10^{-3}, weight decay 10^{-4}), batch size 128 and cross-entropy loss, with global seeding via utils/seed.py. CIFAR-10 runs train for 15 epochs over 3 seeds (42, 123, 999); Tiny ImageNet runs train for 10 epochs with seed 42.

1. Architectural Specifications

The four networks grow from 0.34 M to 57 M parameters; in every one, the dense layers hold most of the parameters while the convolutions carry most of the FLOPs. Output sizes follow the standard formula below (N = input size, K = kernel, P = padding, S = stride), and the receptive field r grows as $r_l = r_{l-1} + (K - 1)\cdot j_{l-1}$, where j is the cumulative stride.

$$N_o = \lfloor (N_I - K + 2P) \,/\, S \rfloor + 1$$

1.1 Base LeNet-5 (Tanh + average pooling)

Layer	Configuration	Output shape	Params	Receptive field (px)
Input	RGB image	3×64×64	0	1
conv1 + Tanh	6 filters, 5×5, s=1, p=0	6×60×60	456	5
avgpool1	2×2, s=2	6×30×30	0	6
conv2 + Tanh	16 filters, 5×5, s=1, p=0	16×26×26	2,416	14

Layer	Configuration	Output shape	Params	Receptive field (px)
avgpool2	2×2, s=2	16×13×13	0	16
flatten	16·13·13	2,704	0	—
fc1 + Tanh	2,704 → 120	120	324,600	—
fc2 + Tanh	120 → 84	84	10,164	—
fc3	84 → 10	10	850	—
Total			**338,486**	

1.2 Modernized LeNet (ReLU + overlapping max pooling)

Padding P = 2 keeps each convolution size-preserving, so only the 3×3, stride-2 pools shrink the map: $\lfloor (64-3)/2 \rfloor + 1 = 31$ and $\lfloor (31-3)/2 \rfloor + 1 = 15$.

Layer	Configuration	Output shape	Params	Receptive field (px)
conv1 + ReLU	6 filters, 5×5, s=1, p=2	6×64×64	456	5
maxpool1	3×3, s=2 (overlapping)	6×31×31	0	7
conv2 + ReLU	16 filters, 5×5, s=1, p=2	16×31×31	2,416	15
maxpool2	3×3, s=2	16×15×15	0	19
flatten	16·15·15	3,600	0	—
fc1 + ReLU	3,600 → 120	120	432,120	—
fc2 + ReLU	120 → 84	84	10,164	—
fc3	84 → 10	10	850	—
Total			**446,006**	

The entire 107,520-parameter increase over LeNet-5 comes from fc1, whose input grows from 2,704 to 3,600 features.

1.3 AlexNet-64 (5 convolutional stages)

Norm = BatchNorm2d after every convolution (bn), LocalResponseNorm(size=5, $\alpha=10^{-4}$, $\beta=0.75$, $k=2$) after conv1 and conv2 only, as in the 2012 paper (lrn), or Identity (none).

Layer	Configuration	Output shape	Params	Receptive field (px)
conv1 + norm + ReLU	64 filters, 5×5, s=2, p=2	64×32×32	4,864	5
maxpool1	3×3, s=2	64×15×15	0	9

Layer	Configuration	Output shape	Params	Receptive field (px)
conv2 + norm + ReLU	192 filters, 5×5, s=1, p=2	192×15×15	307,392	25
maxpool2	3×3, s=2	192×7×7	0	33
conv3 + norm + ReLU	384 filters, 3×3, s=1, p=1	384×7×7	663,936	49
conv4 + norm + ReLU	384 filters, 3×3, s=1, p=1	384×7×7	1,327,488	65
conv5 + norm + ReLU	256 filters, 3×3, s=1, p=1	256×7×7	884,992	81
maxpool3	3×3, s=2	256×3×3	0	97
flatten	256·3·3	2,304	0	—
Dropout(0.5) + fc1 + ReLU	2,304 → 1,024	1,024	2,360,320	—
Dropout(0.5) + fc2 + ReLU	1,024 → 1,024	1,024	1,049,600	—
fc3	1,024 → 10	10	10,250	—
BatchNorm γ, β (bn only)	2 × (64+192+384+384+256)	—	2,560	—
Total			**6,611,402 (bn); 6,608,842 (none, lrn)**	

Convolutions hold 3.19 M parameters (48%) and the classifier 3.42 M (52%). The final receptive field of 97 px exceeds the 64 px input, so every output unit sees the whole image. For Tiny ImageNet (200 classes), fc3 grows to 205,000 parameters and the bn total to 6,806,152. The handout quotes ≈6.3 M; the layer-by-layer sum of the specified architecture is 6.61 M.

1.4 PyTorch Model Zoo AlexNet (torchvision)

The reference model uses an 11×11, stride-4 first convolution designed for 224×224 input. At 64×64 the feature map shrinks 64 → 15 → 7 → 7 → 3 → 3 → 1, and AdaptiveAvgPool2d((6, 6)) then copies each 1×1 value into a 6×6 grid, giving 256·6·6 = 9,216 features that are 36 copies of only 256 distinct numbers.

Block	Output shape at 64×64 input	Params
5 conv layers (64, 192, 384, 256, 256 filters) + 3 max pools	256×1×1	2,469,696
AdaptiveAvgPool2d(6, 6) + flatten	9,216	0
Dropout + fc 9,216 → 4,096 + ReLU	4,096	37,752,832
Dropout + fc 4,096 → 4,096 + ReLU	4,096	16,781,312

Block	Output shape at 64×64 input	Params
fc 4,096 → 10	10	40,970
Total		**57,044,810**

The classifier holds 96% of the parameters, yet the model needs fewer FLOPs than AlexNet-64 because its stride-4 first layer discards spatial resolution immediately.

2. Datasets and Exploratory Data Analysis

Both datasets are perfectly balanced (imbalance ratio 1.00), so Top-1 accuracy is a fair headline metric; Macro F1 is reported beside it to expose weak individual classes. Statistics come from `python -m utils.eda --dataset both`.

Property	CIFAR-10	Tiny ImageNet-200
Classes C	10	200
Train / validation images	50,000 / 10,000	100,000 / 10,000
Training images per class	5,000	500
Native resolution	32×32×3, upscaled to 64×64	64×64×3
Imbalance ratio max N^c / min N^c	1.00	1.00
Entropy ratio H / $\log_2 C$	1.0000 (H = 3.32 bits)	1.0000 (H = 7.64 bits)
Gini impurity $1 - \Sigma p^{c\,2}$	0.900	0.995
Random-guess accuracy	10%	0.5%
Normalize mean (R, G, B)	0.4914, 0.4822, 0.4465	0.4802, 0.4481, 0.3975
Normalize std (R, G, B)	0.2470, 0.2435, 0.2616	0.2764, 0.2689, 0.2821

All tensors enter the network as float32 of shape (B, 3, 64, 64), scaled to [0, 1] by ToTensor and then standardized per channel with the means and standard deviations above.

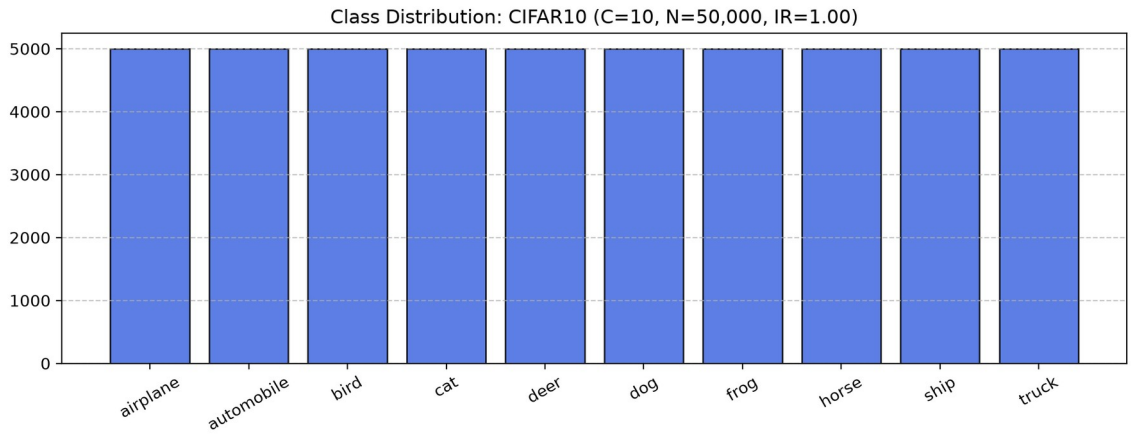


Figure 1a. CIFAR-10 class distribution: 5,000 training images in each of 10 classes (IR = 1.00).

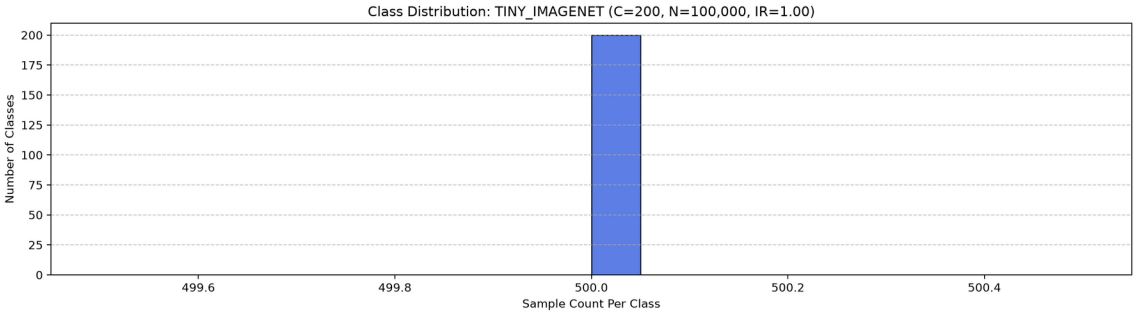


Figure 1b. Tiny ImageNet-200 class distribution: 500 training images in each of 200 classes (IR = 1.00).

The CIFAR-10 channel histograms show the red and green channels almost overlapping, so most colour information is shared brightness. The blue channel carries more mass between 0.1 and 0.35, and all three channels spike at 1.0 from saturated pixels (sky, white backgrounds).

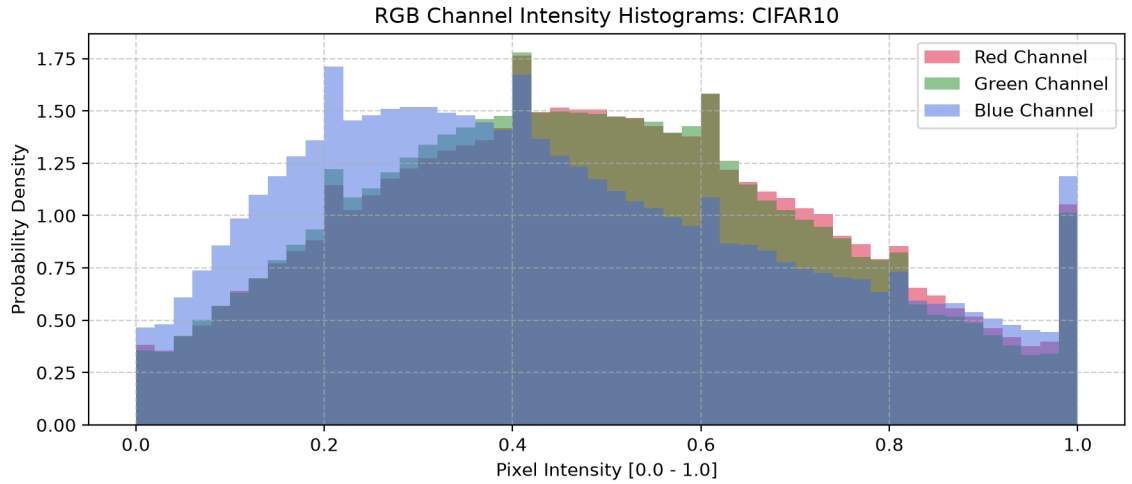


Figure 2. CIFAR-10 RGB intensity histograms (50 bins, probability density).

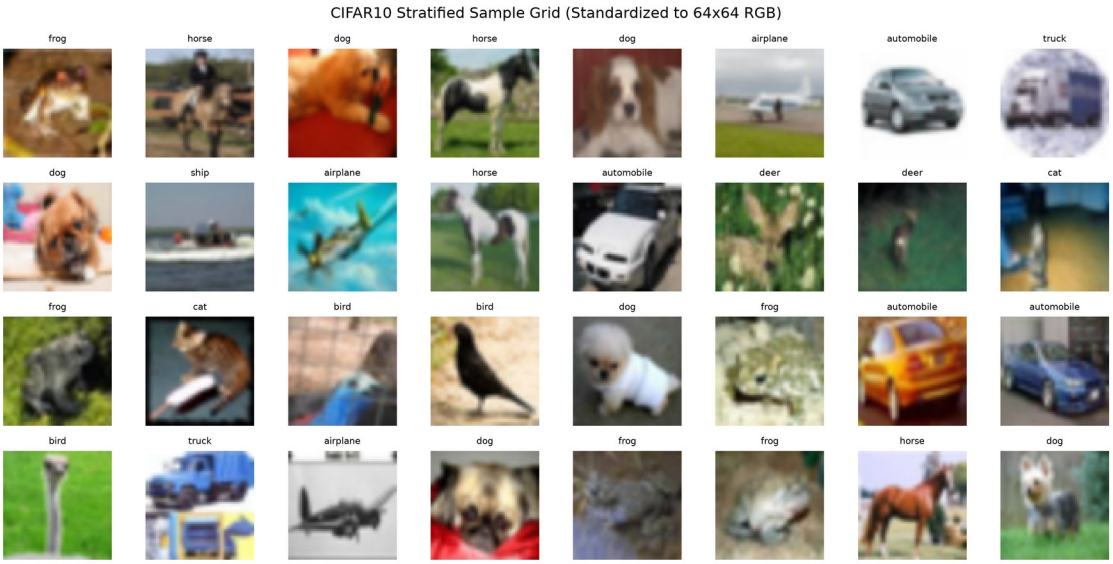


Figure 3. Stratified CIFAR-10 sample grid with ground-truth labels.

```
================================================================================
ELEC 475: Transform Paradigm & Variable Type Diagnostic
================================================================================

[1] OpenCV (cv2) Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Color Space      : Converted BGR -> RGB
    - Memory Location  : Host System RAM (CPU)

[2] NumPy Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Memory Location  : Host System RAM (CPU)

[3] PyTorch (torchvision) Tensor Output:
    - Object Type      : <class 'torch.Tensor'>
    - Data Type (dtype): torch.float32
    - Shape (Layout)   : torch.Size([3, 64, 64])  --> [Channels, Height, Width] (Channels-First)
    - Value Range      : Min = -1.9895, Max = 2.1265 (Standardized)
    - Memory Location  : cuda:0 (NVIDIA GPU VRAM)

--------------------------------------------------------------------------------
Benchmark: Processing 1,000 Images (Resize + Flip + Normalize)
--------------------------------------------------------------------------------
- OpenCV (CPU)            :    9.69 ms  (103,197 img/s) + Host-to-Device Copy required
- NumPy (CPU)             :   30.40 ms  ( 32,889 img/s) + Host-to-Device Copy required
- PyTorch (GPU Tensors)   : 1009.05 ms  (    991 img/s) [ZERO PCIe Transfer Overhead]
================================================================================
```

Figure 4 — screenshot of `python demo_transforms.py` terminal output

3. Empirical Results

Each architectural step improved CIFAR-10 accuracy, from 54.45% (Base LeNet-5) to 81.05% (AlexNet + BN + augmentation), while the 57 M-parameter Zoo AlexNet trailed every custom AlexNet at 63.29%. Values are mean ± sample standard deviation over 3 seeds (n = 3, ddof = 1) for CIFAR-10 and single runs (seed 42) for Tiny ImageNet. Tables are generated by `aggregate_results.py --latency` from the per-run JSON files in results/.

3.1 Table 7.3a — Configuration and computational complexity

Params, FLOPs and latency are measured at batch size 1 on 64×64 input with CIFAR-10 heads (FLOPs = 2 × MACs). Epoch time is the mean over 15 epochs and 3 seeds on cuda (NVIDIA GeForce RTX 5060 Laptop GPU)

#	Model	Conv depth	Normalization and regularization	Augmentation	Params (M)	FLOPs (M)	Mean epoch time (s)	Fastest-seed epoch time (s)	Inference latency (ms)
1	Base LeNet-5	2	None (Tanh + AvgPool)	None	0.338	7.17	5.44 ± 0.09	5.4	0.65
2	Modernized LeNet	2	None (ReLU + overlapping MaxPool)	None	0.446	9.19	5.49 ± 0.11	5.4	0.94
3	AlexNet (No Norm)	5	Dropout (p = 0.5)	None	6.609	436.70	12.58 ± 5.62	8.8	1.27
4	AlexNet + LRN	5	LRN + Dropout (p = 0.5)	None	6.609	436.70	17.73 ± 7.74	12.9	1.90
5	AlexNet + BatchNorm	5	BatchNorm2d + Dropout (p = 0.5)	None	6.611	437.97	14.40 ± 7.00	10.0	1.15
6	AlexNet + BN + Aug	5	BatchNorm2d + Dropout (p = 0.5)	RandomCrop, HFlip, ColorJitter	6.611	437.97	15.84 ± 5.80	12.1	1.19
7	PyTorch Zoo AlexNet	5	Dropout (p = 0.5), AdaptiveAvgPool2d	None	57.045	188.20	17.14 ± 7.88	12.3	1.89

The large epoch-time spread for AlexNet rows comes from the seed-42 pass, which ran about twice as slow for every model (e.g. 22.5 s vs. 10.0 s for BatchNorm), indicating other load on the machine during that pass. The fastest-seed column is therefore the better estimate of each model's true cost. The profiler counts pooling and BatchNorm operations, so its FLOPs (7.17 M, 437 M) exceed the handout's nominal 4.8 M and 250 M; the parameter counts match a layer-by-layer hand calculation (Section 1).

3.2 Table 7.3b — Predictive performance across datasets

Final-epoch values are the weights at the end of training; best-epoch values are the highest validation accuracy reached during training.

#	Model	CIFAR-10 Val Acc, final (%)	CIFAR-10 Val Acc, best epoch (%)	CIFAR-10 Val Macro F1, final (%)	Tiny ImageNet Val Acc (%)	Tiny ImageNet Val Macro F1 (%)
1	Base LeNet-5	54.45 ± 0.93	55.26 ± 0.89	54.29 ± 0.93	12.34	10.48
2	Modernized LeNet	64.99 ± 0.65	66.78 ± 0.56	64.89 ± 0.68	19.72	17.45
3	AlexNet (No Norm)	72.36 ± 1.67	72.70 ± 1.51	71.98 ± 1.78	12.92	10.32
4	AlexNet + LRN	76.46 ± 0.36	76.79 ± 0.37	76.32 ± 0.40	17.65	15.37
5	AlexNet + BatchNorm	79.47 ± 1.35	81.77 ± 0.35	79.31 ± 1.45	**29.88**	**28.47**
6	AlexNet + BN + Aug	**81.05 ± 0.52**	**82.86 ± 1.07**	**80.86 ± 0.60**	28.92	26.96
7	PyTorch Zoo AlexNet	63.29 ± 4.17	63.53 ± 4.29	63.16 ± 4.39	9.30	6.63

Macro F1 tracks accuracy within 0.5 points on CIFAR-10, as expected for a balanced dataset. On Tiny ImageNet the gap widens to 1–3 points, showing that some of the 200 classes are rarely predicted correctly.

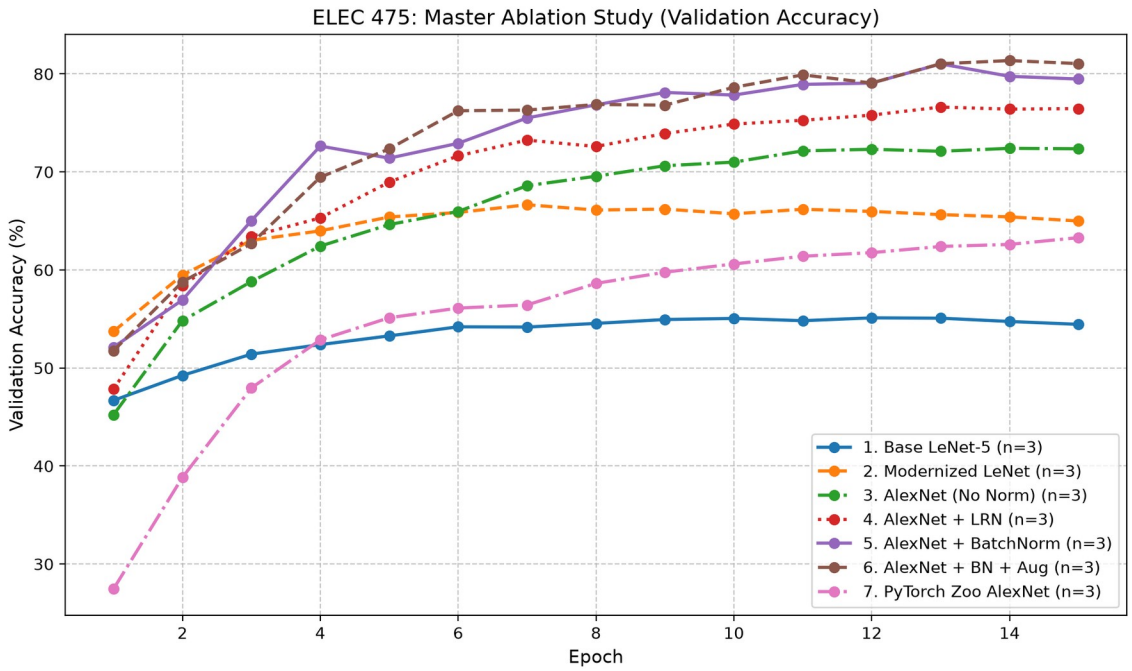


Figure 5. Validation accuracy per epoch for all seven CIFAR-10 configurations (mean of 3 seeds).

```
================================================================================
ELEC 475: Lab 2 - Comprehensive Model Complexity & FLOPs Benchmark
Device        : cuda (NVIDIA GeForce RTX 5060 Laptop GPU)
Dataset       : CIFAR10 (10 classes)
Input Shape   : (1, 3, 64, 64)
================================================================================

Instantiating models for profiling...
Profiling FLOPs, Multiply-Accumulates, and Latency...

| Model Architecture          | Params (M) | MACs (M) | FLOPs (M) | Latency (ms) | Throughput (img/s) |
| ----------------------------|------------|----------|-----------|--------------|------------------- |
| 1. LeNet-5 (Base)           | 0.338      | 3.59     | 7.17      | 0.76         | 1312.6             |
| 2. LeNet-Modern (ReLU+Pool) | 0.446      | 4.59     | 9.19      | 0.76         | 1321.0             |
| 3. AlexNet-64 (None)        | 6.609      | 218.35   | 436.70    | 2.74         | 364.9              |
| 4. AlexNet-64 (LRN)         | 6.609      | 218.35   | 436.70    | 1.95         | 512.4              |
| 5. AlexNet-64 (BatchNorm)   | 6.611      | 218.98   | 437.97    | 3.20         | 312.1              |
| 6. Torchvision Zoo AlexNet  | 57.045     | 94.10    | 188.20    | 2.69         | 371.2              |

================================================================================
Academic Notes on Complexity Metrics:
1. FLOPs vs MACs: 1 Multiply-Accumulate (MAC) corresponds to ~2 Floating Point Operations (FLOPs).
2. Memory vs Compute: Dense Linear layers contribute the majority of parameters, but Conv layers dominate FLOPs.
3. Reference: He et al. (CVPR 2016), Howard et al. (MobileNets 2017).
================================================================================
```

Figure 6 — screenshot of the `python compare_models.py --dataset cifar10 --device cuda`

3.3 Normalization study: none vs. LRN vs. BatchNorm

BatchNorm converged fastest and reached the highest accuracy: it crossed 70% validation accuracy at epoch 4, versus epoch 6 for LRN and epoch 11 for no normalization (seed 42). All three variants were trained without augmentation for 15 epochs.

Variant	Val Acc after epoch 1 (%, s42)	First epoch ≥ 70% val acc (s42)	Final train loss	Final val loss	Final Val Acc (%)	Best Val Acc (%)	Val-loss increases in 14 epoch steps (s42)
None	40.6	11	0.59 ± 0.07	0.83 ± 0.04	72.36 ± 1.67	72.70 ± 1.51	2
LRN	47.6	6	0.46 ± 0.03	0.74 ± 0.02	76.46 ± 0.36	76.79 ± 0.37	4
BatchNorm	49.7	4	0.29 ± 0.00	0.70 ± 0.06	79.47 ± 1.35	81.77 ± 0.35	7

BatchNorm's training loss is the lowest and most consistent across seeds (± 0.00), but its validation loss is the most jagged: it rose in 7 of 14 epoch steps, and its final-epoch accuracy sits 2.3 points below its best epoch. The unnormalized network's validation curve is the smoothest but also the slowest.

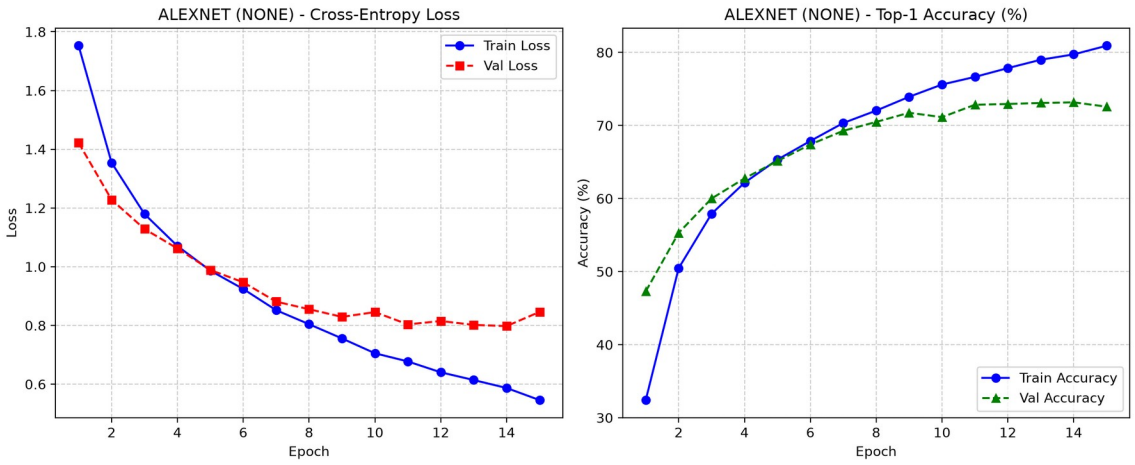


Figure 7a. AlexNet-64, no normalization (seed 42).

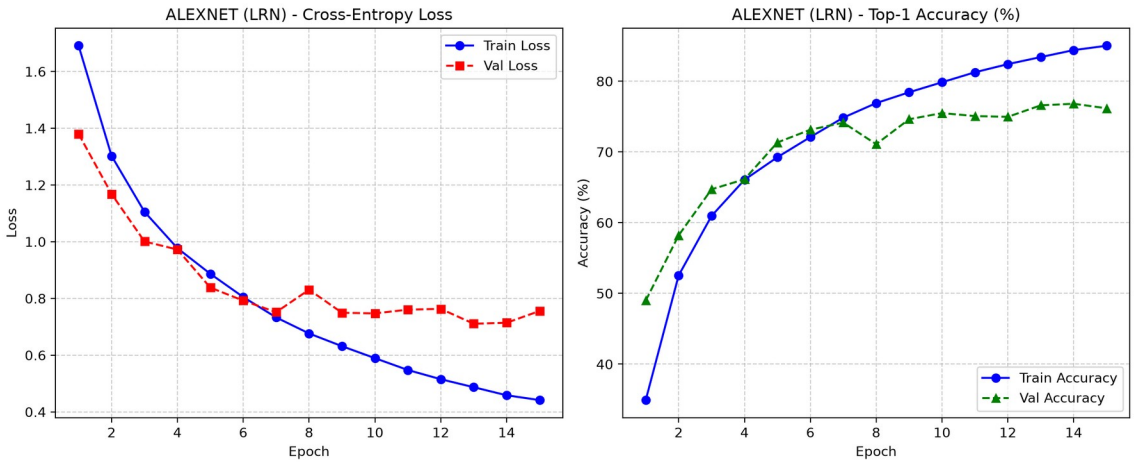


Figure 7b. AlexNet-64 + LRN (seed 42).

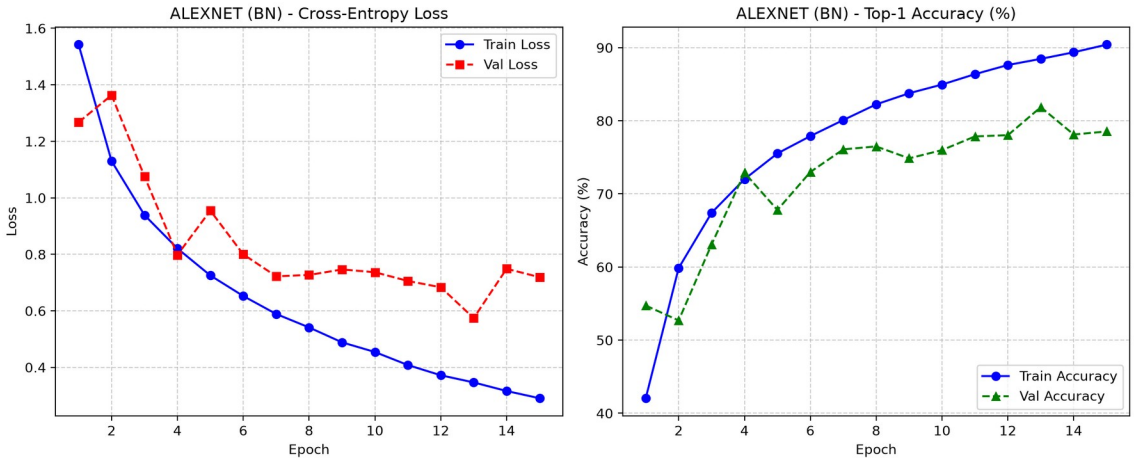


Figure 7c. AlexNet-64 + BatchNorm (seed 42).

3.4 Data augmentation and the generalization gap

Augmentation cut the CIFAR-10 generalization gap by 61%, from 9.95 to 3.92 points, and raised final validation accuracy by 1.58 points. Train accuracy is measured after training in eval mode on the clean (unaugmented) training set, so both rows compare like with like. Δ = train accuracy − validation accuracy.

Dataset	Configuration	Train Acc (%)	Val Acc (%)	Gap Δ (pts)	Val Macro F1 (%)	Final val loss	Epoch of lowest val loss (per seed)
CIFAR-10	AlexNet + BN, no augmentation	89.42 ± 1.47	79.47 ± 1.35	9.95 ± 0.29	79.31 ± 1.45	0.70 ± 0.06	9, 13, 11
CIFAR-10	AlexNet + BN +	84.97 ±	81.05 ±	3.92 ±	80.86 ±	0.57 ±	14, 15, 13

Dataset	Configuration	Train Acc (%)	Val Acc (%)	Gap Δ (pts)	Val Macro F1 (%)	Final val loss	Epoch of lowest val loss (per seed)
	augmentation	0.81	0.52	0.30	0.60	0.02	
Tiny ImageNet	AlexNet + BN, no augmentation	36.42	29.88	6.54	28.47	2.99	10
Tiny ImageNet	AlexNet + BN + augmentation	31.46	28.92	2.54	26.96	3.06	10

Without augmentation the validation loss bottomed out at epochs 9–13 and then rose while training loss kept falling; with augmentation it was still at or near its minimum in the last epochs. Augmentation also made final accuracy 2.6× more consistent across seeds (std 0.52 vs. 1.35) and cost about 2 s extra per epoch (12.1 s vs. 10.0 s on the fastest seed) for CPU-side image transforms.

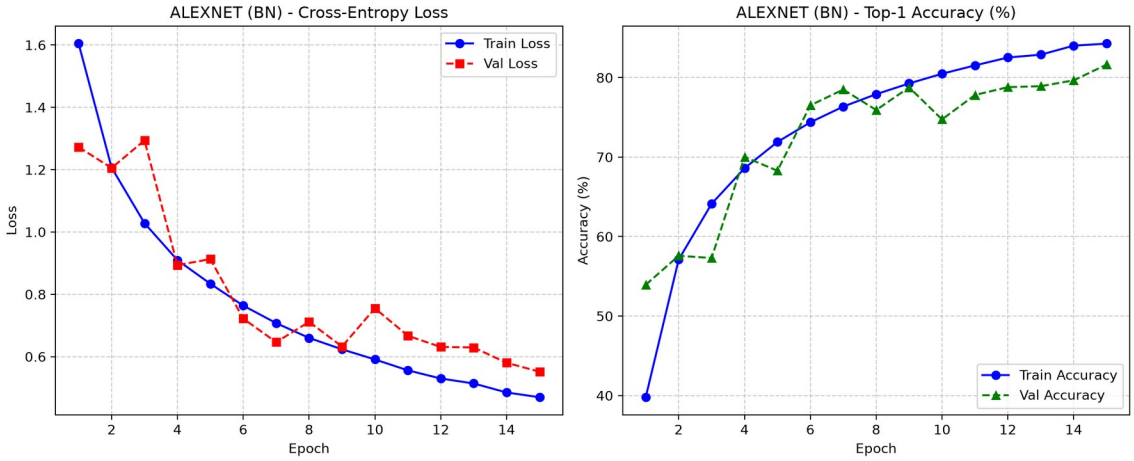


Figure 8. AlexNet-64 + BN + augmentation on CIFAR-10 (seed 42); compare with Figure 7c (no augmentation).

3.5 Tiny ImageNet-200

AlexNet + BN reached 29.88% Top-1 on Tiny ImageNet, 60× the 0.5% random-guess baseline, but 50 points below its CIFAR-10 accuracy. The ranking of models mostly carried over from CIFAR-10, with two exceptions: Modernized LeNet (19.72%) beat both unnormalized (12.92%) and LRN (17.65%) AlexNets, and augmentation slightly lowered accuracy (28.92%).

For every model except Modernized LeNet, the lowest validation loss occurred at the final (10th) epoch, so no Tiny ImageNet run had begun to overfit. The networks were still under-trained when stopped, unlike CIFAR-10, where both LeNets' validation loss bottomed out at epochs 6–9 and then rose.

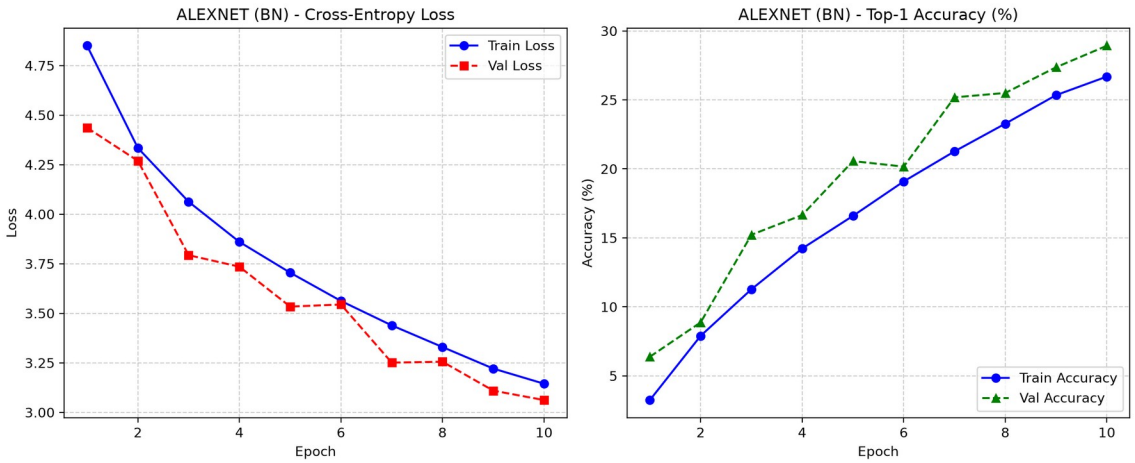


Figure 9. AlexNet-64 + BN + augmentation on Tiny ImageNet-200 (seed 42, 10 epochs).

3.6 PyTorch Model Zoo AlexNet

The 57 M-parameter torchvision AlexNet reached only 63.29 ± 4.17% on CIFAR-10 and 9.30% on Tiny ImageNet, below every custom AlexNet variant on both datasets and level with Modernized LeNet on CIFAR-10. It also had the largest seed-to-seed spread (± 4.17) and sat at chance level (10.0%) after its first epoch for seed 42.

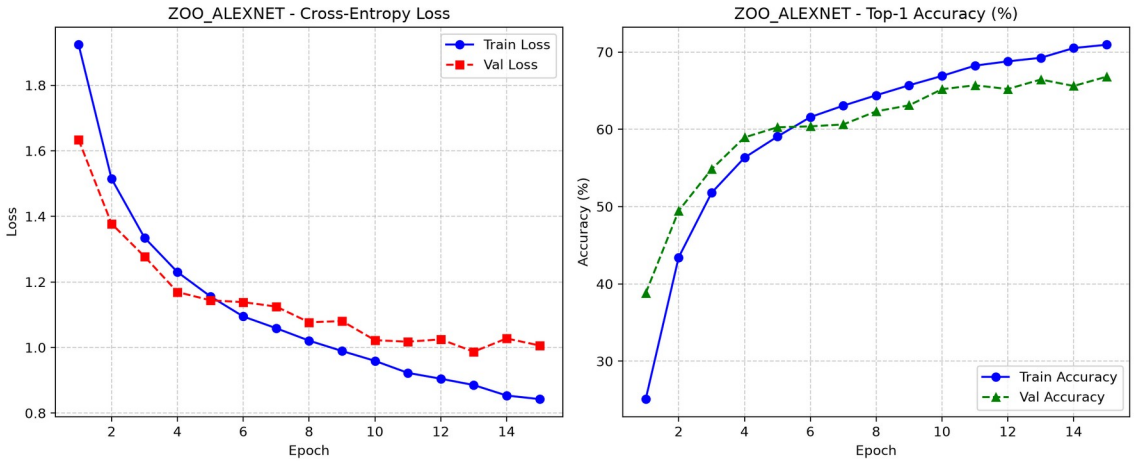


Figure 10. PyTorch Zoo AlexNet on CIFAR-10 (seed 42).

3.7 Multi-seed reproducibility

Every CIFAR-10 configuration was trained with seeds 42, 123 and 999; Tables 7.3a and 7.3b report mean ± sample std. Standard deviations of final accuracy range from 0.36 (LRN) to 4.17 points (Zoo AlexNet). Each step of the ladder from LeNet-5 to AlexNet + BN improves final accuracy by more than twice the larger standard deviation, so those gains are not seed noise. Two comparisons fall within noise: BN vs. BN + augmentation (79.47 ± 1.35 vs. 81.05 ± 0.52 final), where the reliable effect of augmentation is the smaller gap rather than higher accuracy, and Zoo AlexNet vs. Modernized LeNet (63.29 ± 4.17 vs. 64.99 ± 0.65).

3.8 Extended ablation: learning rate and early stopping

Lowering the learning rate to 10^{-4} was the single most effective change tested, adding 3.0 points over the 10^{-3} baseline; raising it to 10^{-2} cost 8.3 points. All runs are AlexNet + BN, CIFAR-10, no augmentation, seed 42; early-stopping runs monitor validation loss with patience 5 and restore the best weights. Generated by results/explore_tables.md.

Experiment	Epochs run	Train Acc (%)	Val Acc (%)	Δ vs. baseline (pts)	Val Macro F1 (%)	Gap Δ (pts)
Baseline: lr 10^{-3}, 15 epochs	15	88.17	78.54	0.00	77.99	9.63
lr 10^{-4}, 15 epochs	15	96.13	81.54	+3.00	81.45	14.59
lr 10^{-4}, early stopping (max 30)	17	95.36	81.86	+3.32	81.62	13.50
lr 10^{-2}, early stopping (max 30)	22	73.08	70.27	−8.27	70.21	2.81

At 10^{-4} the network fit the training set far more closely (96.1% train accuracy) and generalized better, although its gap widened to 14.6 points. Early stopping halted the 10^{-4} run at epoch 17, meaning validation loss bottomed out at epoch 12, and the 10^{-2} run at epoch 22 with the model under-fitting (gap 2.8 points).

4. Technical Analysis and Discussion

4.1 LeNet-5 to Modernized LeNet: ReLU and overlapping max pooling

Yes, the modernized network was clearly better: final CIFAR-10 accuracy rose from 54.45% to 64.99% (+10.5 points) and Macro F1 by the same margin, far outside seed noise (± 0.93). Tanh saturates at ±1, where its derivative $1 - \tanh^2(x)$ approaches zero, so gradients shrink as they pass back through each layer and early filters learn slowly. ReLU's derivative is exactly 1 for positive inputs, so gradients reach conv1 undiminished; this shows in the first epoch alone (Figure 5), where Modernized LeNet already led. Max pooling passes on the strongest local response (an edge or texture peak) instead of averaging it with weaker neighbours, and the 3×3, stride-2 overlap discards less spatial detail per downsampling step.

The cost was 28% more FLOPs (7.17 M → 9.19 M) and 32% more parameters, almost all in fc1 (2,704 → 3,600 inputs). Epoch time did not change (5.44 s vs. 5.49 s) because a network this small cannot saturate a GPU; data loading and per-batch kernel-launch overhead dominate. Single-image latency did rise from 0.65 to 0.94 ms, and on a sequential CPU core or microcontroller, where every FLOP and every byte of SRAM counts, the 1.78 MB weight footprint and extra arithmetic would matter. On a GPU the trade-off is clearly worthwhile.

Both LeNets overfit within the 15-epoch window: validation loss bottomed out at epochs 6–9 and then rose (LeNet-5 to 1.47, Modernized LeNet to 1.34) while training loss kept falling to 0.66 and 0.37. An early-stopping controller with patience 5 would have halted them around epochs 11–14 and restored the epoch-6–9 weights. The in-lab 5-epoch runs (Figures 11a and 11b, Section 5.1) were still improving and would not have stopped.

4.2 Normalization: none vs. LRN vs. BatchNorm

Yes, BatchNorm improved accuracy and Macro F1 over both alternatives: 79.47% vs. 76.46% (LRN) and 72.36% (none) at the final epoch, and 81.77% vs. 76.79% and 72.70% at the best epoch. BatchNorm standardizes each channel over the mini-batch, $\hat{x} = (x - \mu_B) / \sqrt{(\sigma^2_B + \varepsilon)}$, then rescales it with learned γ and β. Every layer therefore sees inputs with stable statistics while earlier layers change, which reduces internal covariate shift and smooths the loss landscape, so each Adam step at lr = 10^{-3} makes more useful progress. This is visible in convergence speed: BatchNorm passed 70% validation accuracy at epoch 4, LRN at epoch 6 and the unnormalized network at epoch 11. LRN only divides each activation by the energy of 5 neighbouring channels at the same pixel; it neither centres the distribution nor learns a scale, so it gives a smaller gain.

BatchNorm's training trajectory was the most stable (final training loss 0.29 ± 0.00 across seeds), but its validation trajectory was the least smooth, rising in 7 of 14 epoch steps and ending 2.3 points below its best epoch. In eval mode BatchNorm uses running estimates of μ and σ^2 (momentum 0.1) that lag behind weights still changing quickly at lr = 10^{-3}, so validation accuracy jumps from epoch to epoch. The extended ablation supports this: at lr = 10^{-4} the same network reached 81.54% at the final epoch. The unnormalized network had the smoothest validation curve simply because it moved slowly.

Normalization barely changes arithmetic: BatchNorm adds 2,560 parameters and 1.27 MFLOPs (0.3%), and LRN adds none. Wall-clock cost tells a different story. On the fastest seed, epochs took 8.8 s (none), 10.0 s (BN, +14%) and 12.9 s (LRN, +47%), and LRN had the highest inference latency (1.90 ms vs. 1.15 ms for BN). LRN's cross-channel power-and-divide is memory-bound and not fused by cuDNN, while BatchNorm is a cheap per-channel affine operation, so BatchNorm delivers by far the better accuracy per second.

4.3 Data augmentation and the generalization gap

Yes, augmentation reduced the generalization gap, from 9.95 ± 0.29 to 3.92 ± 0.30 points on CIFAR-10 (training 89.42% → 84.97%, validation 79.47% → 81.05%), and from 6.54 to 2.54 points on Tiny ImageNet. It also raised final validation accuracy by 1.58 points and Macro F1 by 1.55 points on CIFAR-10, although this gain is within one standard deviation of the unaugmented runs. With 6.6 M parameters and 50,000 images, AlexNet-64 can memorize exact pixel positions. Random crops (±4 px translation), horizontal flips and colour jitter show it a different version of every image each epoch while keeping the label, so filters cannot rely on a fixed position, orientation or colour balance and must learn features invariant to them. The training curves confirm the regularizing effect: unaugmented validation loss turned upward after epochs 9–13, while the augmented run's validation loss was still falling at epochs 13–15, so it would benefit from more than 15 epochs.

4.4 Tiny ImageNet-200 vs. CIFAR-10

Yes, AlexNet learned discriminative features: 29.88% Top-1 is 60× the 0.5% chance level. It is still 50 points below CIFAR-10 because Tiny ImageNet has 20× more classes and 10× fewer training images per class (500 vs. 5,000), many fine-grained categories (several dog breeds, similar vehicles) that differ in small details, and crops from full-resolution ImageNet where objects are often partial or off-centre. Macro F1 trails accuracy by 1.4–2.7 points, against under 0.5 on CIFAR-10, showing that some classes are almost never predicted correctly.

The validation loss did not diverge or plateau earlier than on CIFAR-10; it was still at its minimum at epoch 10 for every model but Modernized LeNet (Figure 9). After 10 epochs the networks are under-trained rather than overfit; the BN model's training loss (2.94) is still far above its CIFAR-10 level. Two results follow from this. Augmentation slightly lowered accuracy (28.92% vs. 29.88%) because it regularizes a model that is not yet overfitting and slows how fast it fits. The deeper but unnormalized AlexNet (12.92%) lost to Modernized LeNet (19.72%) because it converges too slowly to use its capacity in 10 epochs, which again shows how much BatchNorm speeds training.

4.5 Custom AlexNet-64 vs. PyTorch Zoo AlexNet

The Zoo model has 8.6× more parameters (57.0 M) but scored 63.29% on CIFAR-10, 16 points below AlexNet-64 + BN. Its 11×11, stride-4 first convolution is designed for 224×224 input; at 64×64 the feature map collapses to 1×1 before AdaptiveAvgPool2d copies it into a 6×6 grid, so its 37.8 M-parameter fc1 receives only 256 distinct values out of 9,216 inputs. It also has no normalization, which explains its slow start (chance-level accuracy after epoch 1 for seed 42) and its ± 4.17 spread across seeds. Matching the architecture to the input resolution mattered far more than raw parameter count, and the Zoo model's lower FLOPs (188 M) reflect discarded resolution rather than efficiency.

4.6 Learning rate and early stopping

The extended ablation shows that the default lr = 10^{-3} is too high for AlexNet-64 + BN with Adam. At 10^{-4} the network reached 81.54% (+3.0 points) and early stopping found its best weights at epoch 12 (81.86%); at 10^{-2} it under-fit (73.08% train, 70.27% validation) and early stopping halted it at epoch 22. The lower rate also produced a larger generalization gap (14.6 points), so combining lr = 10^{-4} with augmentation is the natural next experiment. Early stopping on validation loss with patience 5 worked as intended: it saved 13 of 30 epochs for the 10^{-4} run with no loss in accuracy.

5. Conclusion

Replacing Tanh and average pooling with ReLU and overlapping max pooling raised CIFAR-10 accuracy from 54.45% to 64.99% at no measurable training-time cost. Scaling to the 5-stage AlexNet-64 with BatchNorm reached 79.47%, ahead of LRN (76.46%) and no normalization (72.36%), and BatchNorm converged nearly three times faster than the unnormalized network. Data augmentation cut the generalization gap from 9.95 to 3.92 points and gave the best model, at 81.05%. On Tiny ImageNet-200 the same network reached 29.88%, 60× chance, showing that class count and samples per class, not depth alone, limit accuracy. The 57 M-parameter Zoo AlexNet's 63.29% shows that an architecture matched to its input resolution matters more than parameter count, and a lower learning rate of 10^{-4} added a further 3 points.

5.1 In-lab training runs (5 epochs)

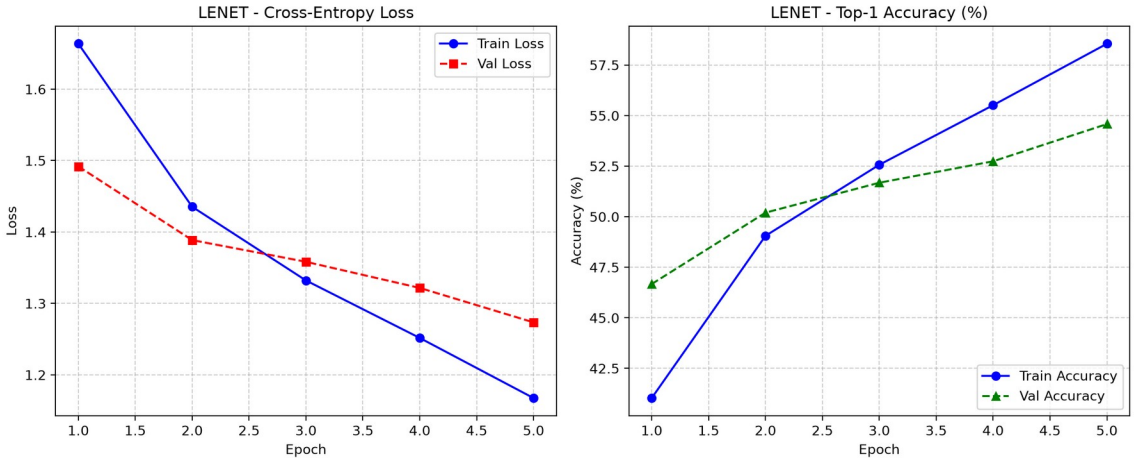


Figure 11a. In-lab Base LeNet-5, 5 epochs, CIFAR-10 (54.6% validation accuracy at epoch 5).

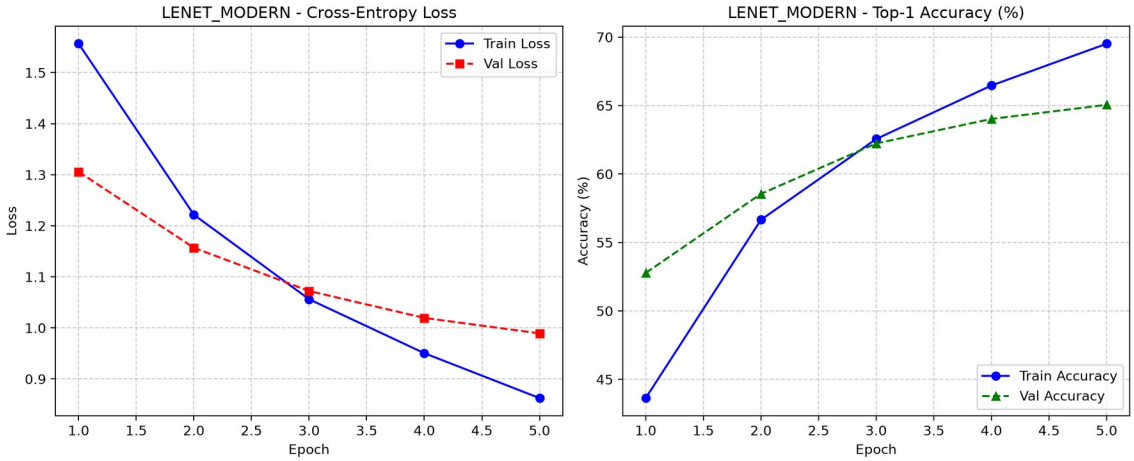


Figure 11b. In-lab Modernized LeNet, 5 epochs, CIFAR-10 (65.0% validation accuracy at epoch 5).

5.2 Implementation notes

- utils/dataset.py: Tiny ImageNet validation reads the official val/ split via val_annotations.txt; the starter template validated on the training images, which would have measured memorization.
- train.py: after training, evaluates clean train accuracy in eval mode for the generalization gap and writes one JSON record per run (metrics, per-epoch history, timing, complexity).
- run_experiments.py runs the 7-model ladder over several seeds; aggregate_results.py builds Tables 7.3a and 7.3b with mean ± sample std via utils/seed.py.
- Normalization-study and Zoo runs use --no-augment so that only row 6 differs in augmentation, matching Table 7.3a.
- Deviations from the handout: CIFAR-10 runs used 15 epochs for every model (the handout suggests 5 for LeNet and 10 for the normalization study) so all seven rows share one training budget.
- Exact commands for every result are in commands.txt; the repository is at **[ADD: GitHub URL]**.